

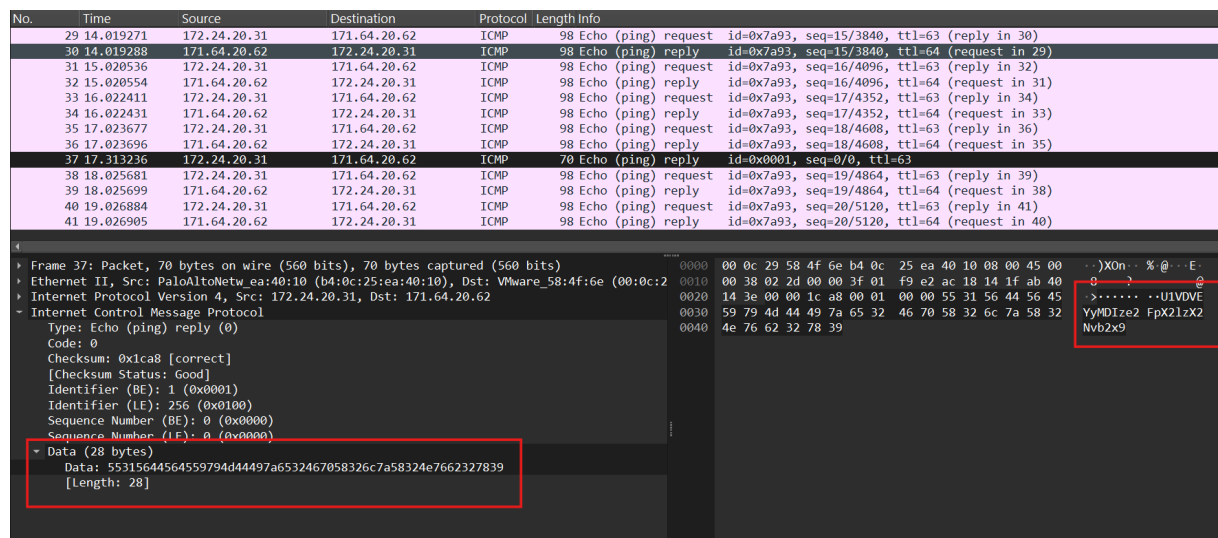
SUBJECT	VULNERABILITY ANALYSIS
CODE	IKB21004 & IKB21403
TASK	FILE ANALYSIS – SCANNING RESULT
NAME & ID	NUR ALIA MAISARAH BINTI ABD RAHAMAN 52215125613 (L01-B05)

Question 1: Analyze packet1.pcap and find the flag.

- Step1: Scan the list of ICMP packets & Identify Packet No.37. Different length than the rest.

32	15.020554	171.64.20.62	172.24.20.31	ICMP	98 Echo (ping) reply	id=0x7a93, seq=16/4096, ttl=64 (request in 31)
33	16.022411	172.24.20.31	171.64.20.62	ICMP	98 Echo (ping) request	id=0x7a93, seq=17/4352, ttl=63 (reply in 34)
34	16.022431	171.64.20.62	172.24.20.31	ICMP	98 Echo (ping) reply	id=0x7a93, seq=17/4352, ttl=64 (request in 33)
35	17.023677	172.24.20.31	171.64.20.62	ICMP	98 Echo (ping) request	id=0x7a93, seq=18/4608, ttl=63 (reply in 36)
36	17.023696	171.64.20.62	172.24.20.31	ICMP	98 Echo (ping) reply	id=0x7a93, seq=18/4608, ttl=64 (request in 35)
37	17.313236	172.24.20.31	171.64.20.62	ICMP	70 Echo (ping) reply	id=0x0001, seq=0/0, ttl=63
38	18.025681	172.24.20.31	171.64.20.62	ICMP	98 Echo (ping) request	id=0x7a93, seq=19/4864, ttl=63 (reply in 39)
39	18.025699	171.64.20.62	172.24.20.31	ICMP	98 Echo (ping) reply	id=0x7a93, seq=19/4864, ttl=64 (request in 38)
40	19.026884	172.24.20.31	171.64.20.62	ICMP	98 Echo (ping) request	id=0x7a93, seq=20/5120, ttl=63 (reply in 41)

- Step2: Inspect packet data
 - Expand the Internet Control Message Protocol
 - Look at Data details; a string of text that looks like random characters:
U1VDVEYyMDIze2FpX2IzX2Nvb2x9



Frame 37: Packet, 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface 0

Ethernet II, Src: PaloAltoNetw_ea:40:10 (b4:0c:25:ea:40:10), Dst: VMware_58:4f:6e (00:0c:29:58:4f:6e)

Internet Protocol Version 4, Src: 172.24.20.31, Dst: 171.64.20.62

Internet Control Message Protocol

Type: Echo (ping) reply (0)

Code: 0

Checksum: 0x1ca8 [correct]

[Checksum Status: Good]

Identifier (BE): 1 (0x0001)

Identifier (LE): 256 (0x0100)

Sequence Number (BE): 0 (0x0000)

Sequence Number (LE): 0 (0x0000)

Data (28 bytes)

Data: 55315644564559794d44497a6532467058326c7a58324e7662327839

[Length: 28]

- Step3: Decode the string
 - Copy that string:- U1VDVEYyMDIze2FpX2IzX2Nvb2x9
 - Use CyberChef to decode:



- SUCTF2023{ai_is_cool} is the flag.

Question 2: Analyse packet2.pcap and find the flag.

- Step1: Analysis the Traffic and identify
 - After inspect the packets (ICMP pings), notice that IPV4 Reserved Bit AKA the Evil Bit being used to hide data.
 - In a normal network, this bit is always 0, but in this file, it flips between 0 and 1 (mine shown as 0x00 and 0x02).
- Step 2: Create a Custom Column in Wireshark
 - To see the hidden bits easily, create a custom column:
 - Go to Edit > Preferences > Appearance > Columns.
 - Click the + button to add a new column.
 - Set the Title to ip.flags.rb and the Fields to ip.flags.rb.
 - Click OK. You will now see a column showing 0x00 and 0x02 for every packet.

No.	Time	Source	Destination	Protocol	Length	Info	ip.flags.rb
1	0.000000	172.24.20.31	172.24.20.62	ICMP	98	Echo (ping) request id=0xc890, seq=1/256, ttl=63 (reply in 2)	0x02
2	0.000020	171.64.20.62	172.24.20.31	ICMP	98	Echo (ping) reply id=0xc890, seq=1/256, ttl=64 (request in 1)	0x00
3	1.001961	172.24.20.31	171.64.20.62	ICMP	98	Echo (ping) request id=0xc890, seq=2/512, ttl=63 (reply in 4)	0x02
4	1.001979	171.64.20.62	172.24.20.31	ICMP	98	Echo (ping) reply id=0xc890, seq=2/512, ttl=64 (request in 3)	0x00
5	2.003244	172.24.20.31	171.64.20.62	ICMP	98	Echo (ping) request id=0xc890, seq=3/768, ttl=63 (reply in 6)	0x02
6	2.003257	171.64.20.62	172.24.20.31	ICMP	98	Echo (ping) reply id=0xc890, seq=3/768, ttl=64 (request in 5)	0x00
7	3.003351	172.24.20.31	171.64.20.62	ICMP	98	Echo (ping) request id=0xc890, seq=4/1024, ttl=63 (reply in 8)	0x02
8	3.003370	171.64.20.62	172.24.20.31	ICMP	98	Echo (ping) reply id=0xc890, seq=4/1024, ttl=64 (request in 7)	0x00
9	4.004365	172.24.20.31	171.64.20.62	ICMP	98	Echo (ping) request id=0xc890, seq=5/1280, ttl=63 (reply in 10)	0x02
10	4.004390	171.64.20.62	172.24.20.31	ICMP	98	Echo (ping) reply id=0xc890, seq=5/1280, ttl=64 (request in 9)	0x00
11	5.006097	172.24.20.31	171.64.20.62	ICMP	98	Echo (ping) request id=0xc890, seq=6/1536, ttl=63 (reply in 12)	0x02
12	5.006117	171.64.20.62	172.24.20.31	ICMP	98	Echo (ping) reply id=0xc890, seq=6/1536, ttl=64 (request in 11)	0x00
13	6.007736	172.24.20.31	171.64.20.62	ICMP	98	Echo (ping) request id=0xc890, seq=7/1792, ttl=63 (reply in 14)	0x02
14	6.007756	171.64.20.62	172.24.20.31	ICMP	98	Echo (ping) reply id=0xc890, seq=7/1792, ttl=64 (request in 13)	0x00
15	6.933722	172.24.20.31	171.64.20.62	TCP	74	59752 → 4444 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2741268109 TSecr=0	0x02
16	6.933749	171.64.20.62	172.24.20.31	TCP	74	4444 → 59752 [ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=2741268109 TSecr=0	0x02
17	6.934794	172.24.20.31	171.64.20.62	TCP	66	59752 → 4444 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2741268109 TSecr=0	0x02
18	7.009493	172.24.20.31	171.64.20.62	ICMP	98	Echo (ping) request id=0xc890, seq=8/2048, ttl=63 (reply in 19)	0x02
19	7.009503	171.64.20.62	172.24.20.31	ICMP	98	Echo (ping) reply id=0xc890, seq=8/2048, ttl=64 (request in 18)	0x00
20	8.010998	172.24.20.31	171.64.20.62	ICMP	98	Echo (ping) request id=0xc890, seq=9/2304, ttl=63 (reply in 21)	0x02
21	8.011015	171.64.20.62	172.24.20.31	ICMP	98	Echo (ping) reply id=0xc890, seq=9/2304, ttl=64 (request in 20)	0x00
22	8.614238	172.24.20.31	171.64.20.62	TCP	72	59752 → 4444 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=6 TSval=274126978 TSecr=0	0x02
23	8.614259	171.64.20.62	172.24.20.31	TCP	66	4444 → 59752 [ACK] Seq=1 Ack=7 Win=65280 Len=0 TSval=3516164885 TSecr=0	0x02
24	9.012126	172.24.20.31	171.64.20.62	ICMP	98	Echo (ping) request id=0xc890, seq=10/2560, ttl=63 (reply in 25)	0x02
25	9.012142	171.64.20.62	172.24.20.31	ICMP	98	Echo (ping) reply id=0xc890, seq=10/2560, ttl=64 (request in 24)	0x00

- Step 3: Export the Data to Excel
 - Since there are hundreds of packets, export the values to a CSV to process them:
 - Go to File > Export Packet Dissections > As CSV...
 - Save the file as test.csv and open it in Excel.
- Step 4: Convert Hexadecimal to Binary
 - In Excel, the Reserved Bit shows as 0x02 for a binary 1 and 0x00 for a binary 0.
 - Use a formula to convert these to simple 1s and 0s: =IF(A2="0x02", 1, 0)
 - Drag this formula down to create a long list of binary digits.
 - Copy the entire column of 1s and 0s.

	A	B	C	D	E	F	G	H	I
1	No.	Time	Source	Destination	Protocol	Length	Info	ip.flags.rb	
2	1	0	172.24.20.	171.64.20.	ICMP	98	Echo (ping 0x02)	1	
3	2	0.00002	171.64.20.	172.24.20.	ICMP	98	Echo (ping 0x00)	0	
4	3	1.001961	172.24.20.	171.64.20.	ICMP	98	Echo (ping 0x02)	1	
5	4	1.001979	171.64.20.	172.24.20.	ICMP	98	Echo (ping 0x00)	0	
6	5	2.003244	172.24.20.	171.64.20.	ICMP	98	Echo (ping 0x02)	1	
7	6	2.003257	171.64.20.	172.24.20.	ICMP	98	Echo (ping 0x00)	0	
8	7	3.003351	172.24.20.	171.64.20.	ICMP	98	Echo (ping 0x02)	1	
9	8	3.00337	171.64.20.	172.24.20.	ICMP	98	Echo (ping 0x00)	0	
10	9	4.004365	172.24.20.	171.64.20.	ICMP	98	Echo (ping 0x02)	1	
11	10	4.00439	171.64.20.	172.24.20.	ICMP	98	Echo (ping 0x00)	0	

- Step 5: Decode with CyberChef
 - Open CyberChef and paste your list of 1s and 0s into the Input box.
 - Add the "Remove whitespace" recipe to turn the list into a single row.
 - Add the "From Binary" recipe (Byte length: 8, Delimiter: None).
- Step 6: Adjust for Bit Alignment (The "Shift")
 - Initially, the output will look like gibberish (ª«×U...) because the capture contains extra "padding" packets at the very beginning.
 - Go to the Input box in CyberChef.
 - Place your cursor at the very start of the numbers.
 - Delete one digit at a time.
 - After deleting the first 32–40 bits (the padding), the flag will suddenly align correctly and appear in the Output box. (Like below)



- The hidden **Flag**: SUCTF2023{bit_by_bit_it_comes}.

Question 3: Interpret an Nmap Output

```
PORT STATE SERVICE VERSION
21/tcp open 6p      vs6pd 2.3.4
22/tcp open ssh      OpenSSH 5.3p1
80/tcp open hBp      Apache 2.2.8
139/tcp open netbios-ssn
445/tcp open microso6-ds Windows 7 Professional 7601 Service Pack 1
```

Questions:

1. What can an attacker do with each port?

Port 21:

- An attacker can attempt to gain unauthorized access to file system. They can try to upload malicious scripts (webshells), download sensitive configuration files or perform a “Brute Force” attack to guess passwords.

Port 22:

- Used to remote administration.
- An attacker can attempt to gain full command-line access to server. They might also refer to use for User Enumeration to find valid usernames on system.

Port 80:

- An attacker can probe the web server for vulnerabilities like SQL Injection, Cross-Site Scripting (XSS), or directory traversal to see files they shouldn't have access to.

Port 139 & 445:

- These ports handle file sharing and printer services.
- An attacker can attempt to access shared folders or, more dangerously, use network-level exploits to take over the machine without needing any login credentials.

2. What vulnerabilities are likely present based on the version?

vsftpd 2.3.4	This specific version is famous for a Backdoor Command Execution vulnerability (CVE-2011-2523). If an attacker logs in with a username ending in :), a shell opens on port 6200.
OpenSSH 5.3p1	This version is very old and likely vulnerable to User Enumeration (CVE-2018-15473), allowing an attacker to confirm which usernames exist on the server.
Apache 2.2.8	This version is susceptible to multiple vulnerabilities, including Denial of Service (DoS) attacks and potential memory leaks.
Windows 7 Professional 7601 Service Pack 1	Because this is an unpatched Windows 7 system, it is highly likely to be vulnerable to EternalBlue (MS17-010) which allows for remote code execution.

3. Which one is the highest risk and why?

- The highest risk is Port 445 (SMB) / Windows 7 SP1.
- Because The EternalBlue vulnerability associated with this version allows an attacker

to gain system-level access remotely without any user interaction or valid credentials. It is a "wormable" exploit, meaning it can be used to spread malware automatically to other computers on the same network.

4. What attack path can be built from this?
 1. **Initial Access:** The attacker uses the vsftpd 2.3.4 backdoor on Port 21 to gain an initial entry point into the system.
 2. **Privilege Escalation:** Once inside, the attacker notices the machine is running Windows 7 SP1. They use the EternalBlue (MS17-010) exploit to upgrade their access to "SYSTEM" (admin) privileges.
 3. **Post-Exploitation:** The attacker steals credentials from the memory and uses the machine as a "pivot" point to attack other computers in the internal network.
5. What should be the remediation?
 - **Immediate Action:** Update vsftpd to a secure version (3.0.x or higher) or replace it with a more secure protocol like SFTP.
 - **Patching:** Apply the MS17-010 security patch to fix the SMB vulnerability.
 - **Upgrade OS:** Because Windows 7 is "End of Life" (no longer receives security updates from Microsoft), the entire operating system should be upgraded to Windows 10 or 11.
 - **Update Services:** Upgrade Apache and OpenSSH to the latest stable versions to close known security holes.
 - **Firewall:** Close Port 21 if it is not strictly necessary for business operations.

Question 4: Identify the OS (OS Fingerprinting) - TTL

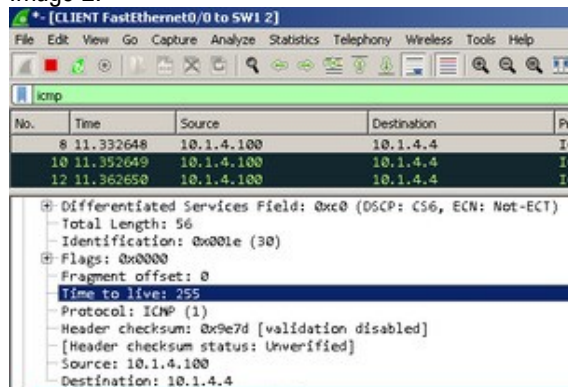
Image 1:

```
[sk@ostechnix ~]$ ping -c5 localhost
PING localhost (localhost (:::1)) 56 data bytes
64 bytes from localhost (:::1): icmp_seq=1 ttl=64 time=0.063 ms
64 bytes from localhost (:::1): icmp_seq=2 ttl=64 time=0.057 ms
64 bytes from localhost (:::1): icmp_seq=3 ttl=64 time=0.070 ms
64 bytes from localhost (:::1): icmp_seq=4 ttl=64 time=0.067 ms
64 bytes from localhost (:::1): icmp_seq=5 ttl=64 time=0.056 ms

--- localhost ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4096ms
rtt min/avg/max/mdev = 0.056/0.062/0.070/0.005 ms
```

Answer: **Linux** because the TTL=64 which standard default for Linux operating system. Window TTL typically be 128.

Image 2:



Answer: **Windows**. TTL=128 is a standard default almost all of the Windows TCP/IP stack.

Image3:

```
64 bytes from 192.168.122.239: icmp_seq=1 ttl=128 time=6.85 ms
64 bytes from 192.168.122.239: icmp_seq=2 ttl=128 time=0.510 ms
64 bytes from 192.168.122.239: icmp_seq=3 ttl=128 time=0.601 ms
64 bytes from 192.168.122.239: icmp_seq=4 ttl=128 time=0.541 ms
64 bytes from 192.168.122.239: icmp_seq=5 ttl=128 time=0.558 ms
```

Answer: **Windows**, the standard default TTL for Windows is 128, the value 127 indicates that the packet has passed through exactly one router (hop) before reaching the capture point.

Question 5: Analyse the Nessus file

Upload to your nessus (Network_Scan.nessus) and analyse the files. Focus on critical or high findings that was identifies in analysis named "Ghostcat".

1. What is the affected Port number?
 - 8009
2. What is the Affected protocol?
 - AJP (Apache JServ Protocol)
3. What is the CVSS Score of vulnerability found?
 - CVSS Score: 9.8 (Critical)
4. Can you find any exploit related to this vulnerability?
 - Yes, it's Metasploit called admin/http/tomcat_ghostcat.
5. Find CVE for this vulnerability.
 - CVE-2020-1938